

SOB4ES - Preparación y procesamiento de datos de ficheros

CRISP-ML(Q) Fase 2: Ingeniería de Datos

Studer et al. (2020): ml-ops.org/content/crisp-ml

Integra todos los conjuntos de datos SOB4ES y las fuentes raster europeas en una única tabla lista para análisis, **una fila por SITE_ID** .

#	Archivo	Hoja(s)	Filas	Clave
1	DD2.2.1_SITE_DESCRIPTIONS	SITE_DETAILS	428	SITE_ID
2	DD2.2.3_ABIOTIC_PHYSICAL	SITE_PHYSICAL, PLOT_PHYSICAL	428 / 1284	SITE_ID
3	DD2.2.4_ABIOTIC_CHEMICAL	CHEM_SITE, CHEM_PLOT	428 / 1346	SITE_ID
4	DD2.2.5.2_ALPHA_DIVERSITIES	ALPHA_DIV_SITE, MICROBIOME_DIV	431 / 447	SITE_ID
5	DD2.2.6_MACROFAUNA_COM	MACROFAUNA_ORDER	1043 parcelas	SITE_ID (agg)
6	DD2.2.7_EARTHWORMS_COM	EARTHWORM_SPECIES_ALL	1107 parcelas	SITE_ID (agg)
7	NUID_UCD_Earthworms	Earthworms - Spatial Sampling	690	SITE_ID (raw)
8	UVIGO_Earthworms	Spain/Slovenia/Romania/Sweden/Israel	~1110	SITE_ID (raw)
9	DD2.2.8_ORIBATIDA_COM	ORIBATID_SPECIES	358	SITE_ID
10	DD2.2.9_MESOTIGMATA_COM	MESOSTIGMATID_SPECIES	491	SITE_ID
11	DD2.2.10_COLLEMBOLA_COM	COLLEMBOLA_SPECIES	362	SITE_ID
12	DD2.2.12_BACTERIA_SEQ	DD2.2.12_BACTERIA_SEQ	447 × 6798 ASVs	SAMPLE_ID
13	DD2.2.13_FUNGI_SEQ	DD2.2.13_FUNGI_SEQ	445 × 14 ASVs	SAMPLE_ID
14	DD2.2.14_EUKARYOTE_SEQ	DD2.2.14_18S_SEQ	447 × 4077 ASVs	SAMPLE_ID
15	DD2.2.15_OOMYCETES_CERCOZOA_SEQ	OOMYCETE_SEQ, CERCOZOAN_SEQ	457	SITE_ID

Capas raster EU (extracción punto a punto por coordenadas):

Capa	Archivo	Resolución	Fuente
Densidad aparente	bulk_density.tif	500 m	ESDAC/LUCAS
Arcilla / Arena / Limo	clay/sand/silt_content.tif	500 m	ESDAC/LUCAS
Textura USDA	soil_texture.tif	500 m	ESDAC/LUCAS
Capacidad de retención hídrica	water_holding_capacity.tif	500 m	ESDAC/LUCAS
CN, K, N, P, pH	CN/K/N/P/pH.tif	500 m	ESDAC/LUCAS
As	LUCAS-median.tif	250 m	ESDAC/LUCAS
Carbono orgánico (OCTOP)	octop_insp.tif	1000 m	ESDAC/OCTOP
Cobre	copper_map_fill.tif	500 m	ESDAC
Níquel / Plomo	Ni_EU27.tif / Pb_EU27.tif	1000 m	ESDAC/LUCAS 2009
Zinc	zinc.tif	1000 m	ESDAC/LUCAS 2009
Zonas ambientales	eu_env_zones_2018_esdac.tif	100 m	EEA 2018
Uso del suelo	eu_land_cover_2018_corine.tif	100 m	CORINE 2018
Tipo de suelo WRB	eu_soil_type_wrb_2006_esdac.tif	1000 m	ESDAC 2006

Estrategia de unión: todas las tablas se unen por **SITE_ID** . Las tablas a nivel de parcela se agregan (media) a nivel de sitio. Los datos de secuenciación (bacteria, eucariotas) se resumen como riqueza + lecturas totales para evitar tablas de ~10 000 columnas.

0.- Configuración del Entorno

En esta sección se configurará el entorno de ejecución como también se harán todas las importaciones necesarias para el correcto funcionamiento del notebook.

```
In [1]: # Importaciones y rutas de datos

import os
import warnings
import numpy as np
import pandas as pd
import openpyxl
import rasterio
from rasterio.crs import CRS
from rasterio.warp import transform
from rasterio.transform import rowcol
from dbfread import DBF
from tqdm.notebook import tqdm
from scipy.stats import entropy as _scipy_entropy

warnings.filterwarnings('ignore')
pd.set_option('display.max_columns', 40)
pd.set_option('display.float_format', '{:.4f}'.format)

# Rutas de datos
DATA_DIR = '../Datasets/SOB4ES_DATASETS/' # datos SOB4ES
EU_DIR = '../Datasets/EU_DATASETS/' # rasters europeos (misma carpeta)
OUT_DIR = 'output/'
os.makedirs(OUT_DIR, exist_ok=True)

WGS84 = CRS.from_epsg(4326) # CRS de las coordenadas SOB4ES
```

```
# Verificación de rutas
for nombre, ruta in [('DATA_DIR', DATA_DIR), ('EU_DIR', EU_DIR), ('OUT_DIR', OUT_DIR)]:
    existe = os.path.isdir(ruta)
    estado = 'Ruta encontrada...' if existe else '[ERROR] Ruta no encontrada, por favor revisa la configuración...'
    print(f' {nombre}: {ruta} {estado}')
print('Entorno configurado correctamente...')

DATA_DIR: ../Datasets/SOB4ES_DATASETS/ Ruta encontrada...
EU_DIR: ../Datasets/EU_DATASETS/ Ruta encontrada...
OUT_DIR: output/ Ruta encontrada...
Entorno configurado correctamente...
```

1.- Descripción de Sitios (*tabla base*)

`SITE_DETAILS` es la tabla base: 428 sitios, una fila cada uno.

Todos los demás conjuntos de datos se unirán a esta tabla mediante `SITE_ID`.

```
In [2]: sites = pd.read_excel(
        DATA_DIR + 'DD2.2.1_SITE_DESCRIPTIONS.xlsx',
        sheet_name='SITE_DETAILS'
    )

# Renombramos las columnas para mayor claridad
sites = sites.rename(columns={
    'Site_latitude' : 'latitude',
    'Site_longitude' : 'longitude',
    'Soil_type_WRB' : 'soil_type',
})

# Parseamos las fechas de muestreo
sites['Sampling_date'] = pd.to_datetime(sites['Sampling_date'], errors='coerce')

print(f'Sites: {sites.shape} | unique SITE_IDs: {sites["SITE_ID"].nunique()}')
print(f'Countries: {sorted(sites["Country"].unique())}')
sites.head(3)
```

Sites: (428, 13) | unique SITE_IDs: 428

Countries: ['BE', 'CH', 'DE', 'ES', 'FR', 'IE', 'IL', 'IT', 'NL', 'RO', 'SE', 'SI']

```
Out[2]:
```

	SITE_ID	SAMPLE_ID	Country	Pedoclimatic_region	Site_locality	latitude	longitude	Sampling_date	soil_type	Land_use_type	Land_use_intensity
0	BE_001	BE_001_DESCRIPTION	BE	ATC	Geel	51.0990	4.9750	2023-10-24	Umbrisol	Grassland	Lov
1	BE_002	BE_002_DESCRIPTION	BE	ATC	Geel	51.1020	4.9780	2023-10-24	Umbrisol	Grassland	Mic
2	BE_003	BE_003_DESCRIPTION	BE	ATC	Geel	51.0970	4.9760	2023-10-27	Arenosol	Grassland	Higl

1.1.- Normalización de nombres

En esta celda se normalizarán todos los nombres que se cargan de base para que tengan una forma de nombrar estándar a lo largo del procesamiento de los consecuentes datasets.

```
In [3]: import re

# Acrónimos que deben preservarse como tokens únicos antes del split CamelCase
_ACRONYM_PROTECT = {
    'pH': 'PH_PLACEHOLDER',
    'CN': 'CN_PLACEHOLDER',
}

_ACRONYM_RESTORE = {v: k.lower() for k, v in _ACRONYM_PROTECT.items()}

def _to_snake(name: str) -> str:
    """
    Convierte cualquier nombre de columna a snake_case limpio:
    - Espacios -> _
    - Acrónimos conocidos (pH, CN) protegidos antes del split
    - CamelCase -> camel_case
    - ALL_CAPS -> todo minúsculas
    - Caracteres no alfanuméricos que no sean _ -> eliminados
    """
    s = name.strip()
    # Proteger acrónimos antes de cualquier transformación
    for token, placeholder in _ACRONYM_PROTECT.items():
        s = s.replace(token, placeholder)
    s = re.sub(r'[s+]', '_', s)
    s = re.sub(r'([a-z0-9])([A-Z])', r'\1_\2', s)
    s = re.sub(r'([A-Z]+)([A-Z][a-z])', r'\1_\2', s)
    s = re.sub(r'^a-zA-Z0-9_', '', s)
    s = s.lower()
    s = re.sub(r'_+', '_', s)
    s = s.strip('_')
    # Restaurar acrónimos protegidos
    for placeholder, restored in _ACRONYM_RESTORE.items():
        s = s.replace(placeholder.lower(), restored)
    return s

# Aplicar a sites en el momento de carga
_sites_rename = {c: _to_snake(c) for c in sites.columns if _to_snake(c) != c}
sites = sites.rename(columns=_sites_rename)
print(f'Columnas renombradas en sites: {len(_sites_rename)}')
for old, new in _sites_rename.items():
    print(f' {old} -> {new}')
```

Columnas renombradas en sites: 10

```
"SITE_ID" → "site_id"
"SAMPLE_ID" → "sample_id"
"Country" → "country"
"Pedoclimatic_region" → "pedoclimatic_region"
"Site_locality" → "site_locality"
"Sampling_date" → "sampling_date"
"Land_use_type" → "land_use_type"
"Land_use_intensity" → "land_use_intensity"
"Dominant_vegetation" → "dominant_vegetation"
"Total_Plant_cover" → "total_plant_cover"
```

2.- Datos Abióticos Físicos

Dos hojas:

- **SITE_PHYSICAL** : una fila por sitio (arcilla, limo, arena, densidad aparente, humedad, estabilidad de agregados).
- **PLOT_PHYSICAL** : tres parcelas por sitio (A/B/C) con columna **SOIL_LAYER** (se agrega a la media del sitio).

```
In [4]: # Datos físicos a nivel de sitio
phys_site = pd.read_excel(
    DATA_DIR + 'DD2.2.3_ABIOTIC_PHYSICAL.xlsx',
    sheet_name='DD2.2.3_SITE_PHYSICAL'
).drop(columns=['SAMPLE_ID'])
# SITE_ID es la clave para unir todo posteriormente

print(f'PHYS_SITE: {phys_site.shape}')
phys_site.head(2)
```

PHYS_SITE: (428, 7)

```
Out[4]:
```

	SITE_ID	clay_content	silt_content	sand_content	aggregate_stability	Bulk density	Soil moisture
0	BE_001	28.5490	35.3960	36.0550	0.9270	0.4233	1.4733
1	BE_002	24.5190	28.9250	46.5560	0.8830	0.8033	0.8400

```
In [5]: # Datos físicos a nivel de parcela (media por sitio)
phys_plot = pd.read_excel(
    DATA_DIR + 'DD2.2.3_ABIOTIC_PHYSICAL.xlsx',
    sheet_name='DD2.2.3_PLOT_PHYSICAL'
)

phys_plot_agg = (
    phys_plot
    .drop(columns=['PLOT_ID', 'SAMPLE_ID'])
    .groupby(['SITE_ID', 'SOIL_LAYER'])
    .mean(numeric_only=True)
    .reset_index()
)

# Pivotar para que cada SOIL_LAYER (M=mineral, O=orgánica) sea un sufijo de columna
phys_plot_pivot = phys_plot_agg.pivot(index='SITE_ID', columns='SOIL_LAYER').round(6)
phys_plot_pivot.columns = [f'plot_{col}_{layer}' for col, layer in phys_plot_pivot.columns]
phys_plot_pivot = phys_plot_pivot.reset_index()

print(f'PHYS_PLOT pivoted: {phys_plot_pivot.shape}')
phys_plot_pivot.head(2)
```

PHYS_PLOT pivoted: (428, 6)

```
Out[5]:
```

	SITE_ID	plot_clay_content_M	plot_silt_content_M	plot_sand_content_M	plot_aggregate_stability_M	plot_Bulk density_M
0	BE_001	28.5490	35.3960	36.0550	0.9270	0.4265
1	BE_002	24.5190	28.9250	46.5560	0.8830	0.8060

3.- Datos Abióticos Químicos

- **CHEM_SITE** : metales pesados + pH por sitio.
- **CHEM_PLOT** : Total_C, Total_organic_C, Total_N por parcela (se agrega a la media del sitio).

```
In [6]: # Química a nivel de sitio
chem_site = pd.read_excel(
    DATA_DIR + 'DD2.2.4_ABIOTIC_CHEMICAL.xlsx',
    sheet_name='DD2.2.4_CHEM_SITE'
).drop(columns=['SAMPLE_ID'])

# Química a nivel de parcela (media por sitio)
chem_plot = pd.read_excel(
    DATA_DIR + 'DD2.2.4_ABIOTIC_CHEMICAL.xlsx',
    sheet_name='DD_2.4_CHEM_PLOT'
)

chem_plot_agg = (
    chem_plot
    .drop(columns=['PLOT_ID'])
    .groupby('SITE_ID')
    .mean(numeric_only=True)
    .add_prefix('plot_')
    .reset_index()
)

# Resumen de los datos obtenidos
print(f'CHEM_SITE: {chem_site.shape}')
print(f'Columns: {list(chem_site.columns)}')
display(chem_site.head(3))

print(f'CHEM_PLOT aggregated: {chem_plot_agg.shape}')
display(chem_plot_agg.head(3))
```

CHEM_SITE: (428, 10)

Columns: ['SITE_ID', 'As', 'Cu', 'K', 'Mo', 'Ni', 'P', 'Pb', 'Zn', 'soil_pH']

	SITE_ID	As	Cu	K	Mo	Ni	P	Pb	Zn	soil_pH
0	BE_001	159.0000	15.0000	8.7137	0.0000	0	6.5939	93.0000	129.0000	4.5000
1	BE_002	53.3000	14.4000	7.5104	4.0000	0	7.0087	47.3000	79.0000	4.9100
2	BE_003	6.2000	11.9000	3.7344	4.4000	0	8.3843	32.6000	47.5000	4.4800

CHEM_PLOT aggregated: (428, 4)

	SITE_ID	plot_Total_C	plot_Total_organic_C	plot_Total_N
0	BE_001	12.8458	12.8458	0.9610
1	BE_002	10.6648	10.6648	0.8655
2	BE_003	6.9343	6.9343	0.6308

4.- Índices de Diversidad Alfa

Dos hojas con nombres de columna clave distintos:

- ALPHA_DIV_SITE usa SITE_ID .
- MICROBIOME_DIV_SITE usa SampleID (usa los mismos valores de SITE-ID, por lo tanto se renombran).

```
In [7]: alpha_div = pd.read_excel(
        DATA_DIR + 'DD2.2.5.2_ALPHA_DIVERSITIES.xlsx',
        sheet_name='ALPHA_DIV_SITE'
    )
    # Normalizar nombres a snake_case
    alpha_div.columns = [_to_snake(c) for c in alpha_div.columns]
    # Conservar solo SITE_ID + nematode_shannon (sin cálculo propio disponible)
    _alpha_keep = [c for c in alpha_div.columns
                   if c.lower() == 'site_id' or 'nematode' in c.lower()]
    alpha_div = alpha_div[_alpha_keep].copy()

    micro_div = pd.DataFrame() # placeholder vacío, no se fusiona

    print(f'ALPHA_DIV (solo nematode_shannon): {alpha_div.shape}')
    print(f'Alpha cols: {list(alpha_div.columns)}')
```

ALPHA_DIV (solo nematode_shannon): (431, 2)

Alpha cols: ['site_id', 'nematode_shannon']

5.- Comunidad de Macrofauna

Conteos a nivel de parcela por orden (se agrega por sitio): abundancia media por orden + abundancia total + riqueza de órdenes.

```
In [8]: macro = pd.read_excel(
        DATA_DIR + 'DD2.2.6_MACROFAUNA_COM.xlsx',
        sheet_name='MACROFAUNA_ORDER'
    )

    # Normalizar nombres de columna del Excel antes de cualquier operación
    macro.columns = [_to_snake(c) for c in macro.columns]

    id_cols_macro = ['site_id', 'plot_id', 'sample_id']
    macro_taxa_cols = [c for c in macro.columns if c not in id_cols_macro]

    # Convertir a numérico (el archivo puede contener '-', 'nd', celdas vacías)
    macro[macro_taxa_cols] = macro[macro_taxa_cols].apply(
        pd.to_numeric, errors='coerce'
    ).fillna(0)

    # Fusionar Aranea y Araneida → Araneae (mismo taxón, dos nombres en la fuente)
    if 'aranea' in macro.columns and 'araneida' in macro.columns:
        macro['araneae'] = macro['aranea'] + macro['araneida']
        macro = macro.drop(columns=['aranea', 'araneida'])
        macro_taxa_cols = [c for c in macro.columns if c not in id_cols_macro]

    macro_agg = (
        macro
        .drop(columns=['plot_id', 'sample_id'], errors='ignore')
        .groupby('site_id')[macro_taxa_cols]
        .mean()
        .add_prefix('macro_')
    )

    def _shannon_matrix(df_taxa: pd.DataFrame) → pd.Series:
        """
        Computes Shannon H' (natural log base) row-wise from an abundance matrix.
        Rows with zero total abundance get H' = 0.
        """
        mat = df_taxa.astype(float)
        totals = mat.sum(axis=1)
        prob = mat.div(totals.where(totals > 0, 1), axis=0)
        return prob.apply(
            lambda row: _scipy_entropy(row.values, base=np.e) if row.sum() > 0 else 0.0,
            axis=1
        )

    taxa_prefixed = list(macro_agg.columns)
    macro_agg['macro_total_abundance'] = macro_agg[taxa_prefixed].sum(axis=1)
    macro_agg['macro_order_richness'] = (macro_agg[taxa_prefixed] > 0).sum(axis=1)
    macro_agg['macro_shannon'] = _shannon_matrix(macro_agg[taxa_prefixed])
    macro_agg = macro_agg.reset_index()

    print(f'MACROFAUNA agregado: {macro_agg.shape}')
    print(f' Taxa cols: {len(taxa_prefixed)} (20, Araneae fusionada)')
```

```
print(f' Sitios con abundancia > 0: {(macro_agg["macro_total_abundance"] > 0).sum()}')
print(f' Sitios con abundancia = 0: {(macro_agg["macro_total_abundance"] == 0).sum()}')
print(f' macro_shannon min={macro_agg["macro_shannon"].min():.4f} '
      f'max={macro_agg["macro_shannon"].max():.4f} '
      f'ceros={macro_agg["macro_shannon"]==0).sum()}')
macro_agg.head(3)
```

MACROFAUNA agregado: (368, 26)
 Taxa cols: 22 (20, Araneae fusionada)
 Sitios con abundancia > 0: 358
 Sitios con abundancia = 0: 10
 macro_shannon min=0.0000 max=2.8307 ceros=29

```
Out[8]:
```

	site_id	macro_amphipoda	macro_blattodea	macro_chilopoda	macro_coleoptera	macro_dermaptera	macro_diplopoda	macro_diplura	macro_diptera	macro_hemiptera
0	BE_001	0.0000	0.0000	0.3333	2.3333	0.0000	0.3333	0.0000	0.3333	0.0000
1	BE_002	0.0000	0.0000	0.0000	4.0000	0.0000	0.0000	0.0000	0.6667	0.0000
2	BE_003	0.0000	0.0000	0.0000	1.0000	0.0000	0.0000	0.0000	0.0000	0.0000

6.- Comunidad de Lombrices

6.1.- Archivos RAW

6.1.1.- Lombrices RAW NUID

Cargamos y procesamos los datos provenientes de NUID.

```
In [9]: print("Procesando datos crudos de Lombrices (NUID)...")

nuid_raw = pd.read_excel(
    DATA_DIR + 'EARTHWORMS_RAW/NUID_UCD_Earthworms.xlsx',
    sheet_name='Earthworms - Spatial Sampling',
    header=None
)

# Extraer cabeceras reales (Fila 1) y limpiar el dataframe
cols_nuid = ['sample_id'] + nuid_raw.iloc[1, 1:].tolist()
nuid_raw.columns = cols_nuid
nuid_raw = nuid_raw.iloc[2:].reset_index(drop=True)

# Extraer SITE_ID (ej. de 'IE_001_A_M_W' a 'IE_001')
nuid_raw['site_id'] = nuid_raw['sample_id'].astype(str).str.extract(r'([A-Z]{2}_\d{3})')
nuid_raw = nuid_raw.dropna(subset=['site_id'])

# Aislamos las columnas de conteo de especies (Ignorando Biomasa y Riqueza pre-calculada)
start_idx = cols_nuid.index('Fragment')
taxa_cols_nuid = cols_nuid[start_idx:]

# Forzar a numérico y agrupar por sitio (promedio de las parcelas para crear un perfil de comunidad)
nuid_raw[taxa_cols_nuid] = nuid_raw[taxa_cols_nuid].apply(
    pd.to_numeric, errors='coerce'
).fillna(0)
nuid_site = nuid_raw.groupby('site_id')[taxa_cols_nuid].mean()

# Calcular métricas puras NUID
nuid_summary = pd.DataFrame(index=nuid_site.index)
nuid_summary['earthworm_abundance'] = nuid_site.sum(axis=1)
nuid_summary['earthworm_density_m2'] = nuid_summary['earthworm_abundance'] * 16
nuid_summary['earthworm_richness'] = (nuid_site > 0).sum(axis=1)
nuid_summary['earthworm_shannon'] = _shannon_matrix(nuid_site)

print(f"NUID procesado: {len(nuid_summary)} sitios listos...")
display(nuid_summary.head(3))
```

Procesando datos crudos de Lombrices (NUID)...

NUID procesado: 218 sitios listos...

	earthworm_abundance	earthworm_density_m2	earthworm_richness	earthworm_shannon
site_id				
BE_001	10.3333	165.3333	5	1.4187
BE_002	18.3333	293.3333	6	1.2276
BE_003	6.6667	106.6667	5	1.4887

6.1.2.- Lombrices RAW UVIGO

Cargamos y procesamos los datos de lombrices provenientes de la UVIGO.

```
In [10]: print("Procesando datos crudos de Lombrices (UVIGO)...")

uvigo_sheets = ['Spain', 'Slovenia', 'Romania', 'Sweden', 'Israel']
uvigo_frames = []
for sheet in uvigo_sheets:
    df = pd.read_excel(DATA_DIR + 'EARTHWORMS_RAW/UVIGO_Earthworms.xlsx', sheet_name=sheet)
    uvigo_frames.append(df)

uvigo_all = pd.concat(uvigo_frames, ignore_index=True)

# Extraer site_id (ej. de 'ES_001_A' a 'ES_001')
uvigo_all['site_id'] = uvigo_all['Sample ID'].astype(str).str.extract(r'([A-Z]{2}_\d{3})')
uvigo_all = uvigo_all.dropna(subset=['site_id'])

# Usar estrictamente el conteo físico ('Total No. Individuals') para evitar densidades escaladas en la diversidad
uvigo_all['Total No. Individuals'] = pd.to_numeric(
    uvigo_all['Total No. Individuals'], errors='coerce'
).fillna(0)
```

```
# Etiquetar conteos sin especie adulta (ej. Juveniles) para no perderlos en el pivot
uvigo_all['Species'] = uvigo_all['Species'].fillna('Unknown_Taxa')

# Pivotar para crear la matriz de Abundancia: Parcelas x Especies
uvigo_pivot = uvigo_all.pivot_table(
    index=['site_id', 'Sample ID'],
    columns='Species',
    values='Total No. Individuals',
    aggfunc='sum',
    fill_value=0
).reset_index()

taxa_cols_uvigo = [c for c in uvigo_pivot.columns if c not in ['site_id', 'Sample ID']]

# Agrupar por sitio (promedio de las parcelas para el perfil comunitario)
uvigo_site = uvigo_pivot.groupby('site_id')[taxa_cols_uvigo].mean()

# Calcular métricas puras UVIGO
uvigo_summary = pd.DataFrame(index=uvigo_site.index)
uvigo_summary['earthworm_abundance'] = uvigo_site.sum(axis=1) # Individuos por monolito
uvigo_summary['earthworm_density_m2'] = uvigo_summary['earthworm_abundance'] * 16 # Individuos por m2
uvigo_summary['earthworm_richness'] = (uvigo_site > 0).sum(axis=1)
uvigo_summary['earthworm_shannon'] = _shannon_matrix(uvigo_site)

print(f" UVIGO procesado: {len(uvigo_summary)} sitios listos...")
display(uvigo_summary.head(3))
```

Procesando datos crudos de Lombrices (UVIGO)...

UVIGO procesado: 162 sitios listos...

	earthworm_abundance	earthworm_density_m2	earthworm_richness	earthworm_shannon
site_id				
ES_001	1.6667	26.6667	2	0.5004
ES_002	1.0000	16.0000	2	0.6365
ES_003	1.6667	26.6667	3	0.9503

6.2.- Consolidación de datos

Una vez procesados los datos de earthworms (UVIGO y NUID), procederemos a la consolidación de los datos en una única base.

```
In [11]: ew_summary_final = pd.concat([nuid_summary, uvigo_summary]).reset_index()

# Prevenir duplicados (si un sitio apareciera en ambas bases por error, mantenemos el primero).
ew_summary_final = ew_summary_final.drop_duplicates(subset=['site_id'], keep='first')

# Eliminamos columnas duplicadas/innecesarias
ew_summary_final = ew_summary_final.drop(columns=['earthworm_density_m2'], errors='ignore')

print(f'Matriz maestra de lombrices consolidada: {ew_summary_final.shape[0]} sitios')
print(f' Columnas: {list(ew_summary_final.columns)}')
```

Matriz maestra de lombrices consolidada: 380 sitios

Columnas: ['site_id', 'earthworm_abundance', 'earthworm_richness', 'earthworm_shannon']

6.3.- Verificación con DD2.2.7

```
In [12]: print("Ejecutando verificación cruzada de Lombrices...")

# 1. Cargar el archivo combinado antiguo (solo para comparar)
ew_com_old = pd.read_excel(
    DATA_DIR + 'DD2.2.7_EARTHWORMS_COM.xlsx',
    sheet_name='EARTHWORM_SPECIES_ALL'
)

# Sacar las columnas de especies y forzar a numérico (tolerante a mayúsculas/minúsculas)
ew_old_species = [c for c in ew_com_old.columns if c.lower() not in ['site_id', 'plot_id', 'sample_id']]
ew_com_old[ew_old_species] = ew_com_old[ew_old_species].apply(pd.to_numeric, errors='coerce').fillna(0)

# Detectar dinámicamente el nombre exacto de la columna de sitio en ew_com_old para evitar KeyError
site_col_old = [c for c in ew_com_old.columns if c.lower() == 'site_id'][0]

# Calcular la vieja abundancia agregada por sitio
ew_old_agg = ew_com_old.groupby(site_col_old)[ew_old_species].sum().sum(axis=1).reset_index(name='Original_Abundance')

# Asegurarnos de que ew_old_agg tiene la columna con el nombre estándar 'site_id' en minúsculas
ew_old_agg = ew_old_agg.rename(columns={site_col_old: 'site_id'})

# 2. CORRECCIÓN DE TIPOS (Evita ValueError al hacer el merge):
# Homogeneizar ambos dataframes a tipo texto (string) y limpiar decimales flotantes como '.0'
ew_summary_final['site_id'] = ew_summary_final['site_id'].astype(str).str.replace('.0', '', regex=False)
ew_old_agg['site_id'] = ew_old_agg['site_id'].astype(str).str.replace('.0', '', regex=False)

# 3. Cruzar usando los nombres exactos en minúsculas y tipos de datos compatibles
check_df = ew_summary_final[['site_id', 'earthworm_abundance']].merge(ew_old_agg, on='site_id', how='inner')

# 4. Calcular la diferencia (Delta) usando los nombres correctos
check_df['Diferencia_Individuos'] = check_df['Original_Abundance'] - check_df['earthworm_abundance']

# 5. Mostrar los resultados de la auditoría
print("\n Resultados de las verificaciones: ")
sitios_con_error = check_df[check_df['Diferencia_Individuos'].abs() > 0.01]

print(f"Sitios totales comparados: {len(check_df)}")
print(f"Sitios donde la abundancia antigua estaba INFLADA o MAL: {len(sitios_con_error)}")

if len(sitios_con_error) > 0:
    print("\nEjemplo de los peores desfases (Muestra de 5 sitios):")
```

```
# Mostrar los 5 con mayor diferencia
display(sitios_con_error.sort_values('Diferencia_Individuos', ascending=False).head())
else:
    print("\n¡Sorpresa! Ambos archivos cuadran perfectamente.")
```

Ejecutando verificación cruzada de Lombrices...

Resultados de las verificaciones:
 Sitios totales comparados: 298
 Sitios donde la abundancia antigua estaba INFLADA o MAL: 268

Ejemplo de los peores desfases (Muestra de 5 sitios):

	site_id	earthworm_abundance	Original_Abundance	Diferencia_Individuos
229	ES_013	65.3333	3293.0000	3227.6667
273	SE_003	20.6667	1403.0000	1382.3333
224	ES_008	28.0000	1400.0000	1372.0000
281	SE_011	20.6667	1353.0000	1332.3333
282	SE_012	16.3333	1320.0000	1303.6667

7.- Comunidades de Ácaros del Suelo y Colémbolos

Los tres archivos tienen columna `SOIL_LAYER` (`M` = mineral, `O` = orgánica).

Estrategia: se usa solo la capa mineral (`M`) para una comparación consistente entre sitios, y se resume como riqueza + abundancia total (las columnas de especies son muy anchas para unir las directamente: Oribátida tiene 329 spp., Mesostigmata 205, Colémbolos 115).

```
In [13]: def summarise_community(filepath, sheet, id_cols, layer_col=None,
    prefix='', layer='M'):
    """
    Carga una hoja de abundancia comunitaria y devuelve un resumen por sitio:
    - {prefix}total_abundance : suma de conteos reales de especies
    - {prefix}species_richness : número de especies con al menos 1 individuo
    - {prefix}shannon : H' (ln) calculado desde la matriz de abundancia
    Filtra por capa de suelo e ignora columnas Unnamed y totales de Excel.
    """
    df = pd.read_excel(filepath, sheet_name=sheet)

    if layer_col and layer_col in df.columns:
        df = df[df[layer_col] == layer]

    # Excluir IDs, capas, columnas Unnamed y totales precalculados
    species_cols = [
        c for c in df.columns
        if c not in id_cols + ([layer_col] if layer_col else [])
        and c is not None
        and 'Unnamed' not in str(c)
        and 'total' not in str(c).lower()
        and 'sum' not in str(c).lower()
        and 'nymph' not in str(c).lower()
    ]

    df[species_cols] = df[species_cols].apply(pd.to_numeric, errors='coerce').fillna(0)

    summary = df.groupby('SITE_ID')[species_cols].mean(numeric_only=True)

    summary[prefix + 'total_abundance'] = summary[species_cols].sum(axis=1)
    summary[prefix + 'species_richness'] = (summary[species_cols] > 0).sum(axis=1)
    summary[prefix + 'shannon'] = _shannon_matrix(summary[species_cols])

    return summary[[
        prefix + 'total_abundance',
        prefix + 'species_richness',
        prefix + 'shannon',
    ]].reset_index()

# 1. Oribátida (capa mineral)
orib_sum = summarise_community(
    DATA_DIR + 'DD2.2.8_ORIBATIDA.COM.xlsx',
    sheet='ORIBATID_SPECIES',
    id_cols=['SITE_ID', 'SAMPLE_ID'], layer_col='SOIL_LAYER',
    prefix='orib_', layer='M'
)

# 2. Mesostigmata (capa mineral)
meso_sum = summarise_community(
    DATA_DIR + 'DD2.2.9_MESOTIGMATA.COM.xlsx',
    sheet='MESOTIGMATID_SPECIES',
    id_cols=['SITE_ID', 'SAMPLE_ID'], layer_col='SOIL_LAYER',
    prefix='meso_', layer='M'
)

# 3. Colémbolos - ninfas aisladas, Shannon sólo de adultos COLL_*
df_coll_raw = pd.read_excel(
    DATA_DIR + 'DD2.2.10_COLLEMBOLA.COM.xlsx', sheet_name='COLLEMBOLA_SPECIES'
)
df_coll_raw = df_coll_raw[df_coll_raw['Soil Layer'] == 'M'].copy()

coll_species_cols = [c for c in df_coll_raw.columns if str(c).startswith('COLL_')]
df_coll_raw[coll_species_cols] = df_coll_raw[coll_species_cols].apply(
    pd.to_numeric, errors='coerce'
).fillna(0)
df_coll_raw['NYMPHS'] = pd.to_numeric(df_coll_raw['NYMPHS'], errors='coerce').fillna(0)

coll_site_spp = df_coll_raw.groupby('SITE_ID')[coll_species_cols].mean()
coll_site_nymphs = df_coll_raw.groupby('SITE_ID')['NYMPHS'].mean()
```

```
coll_sum = pd.DataFrame(index=coll_site_spp.index)
coll_sum['coll_total_abundance'] = coll_site_spp.sum(axis=1)
coll_sum['coll_species_richness'] = (coll_site_spp > 0).sum(axis=1)
coll_sum['coll_nymphs_abundance'] = coll_site_nymphs
# Shannon from adult species matrix only (nymphs excluded - no species resolution)
coll_sum['coll_shannon'] = _shannon_matrix(coll_site_spp)
coll_sum = coll_sum.reset_index()

print(f'Oribatida: {orib_sum.shape} | Mesostigmata: {meso_sum.shape} | Collembola: {coll_sum.shape}')
print(f' orib_shannon min={orib_sum["orib_shannon"].min():.4f} '
      f'max={orib_sum["orib_shannon"].max():.4f}')
print(f' meso_shannon min={meso_sum["meso_shannon"].min():.4f} '
      f'max={meso_sum["meso_shannon"].max():.4f}')
print(f' coll_shannon min={coll_sum["coll_shannon"].min():.4f} '
      f'max={coll_sum["coll_shannon"].max():.4f}')
coll_sum.head(3)
```

```
Oribatida: (308, 4) | Mesostigmata: (433, 4) | Collembola: (334, 5)
orib_shannon min=0.0000 max=2.9818
meso_shannon min=0.0000 max=2.0947
coll_shannon min=0.0000 max=2.2550
```

```
Out[13]:
```

	SITE_ID	coll_total_abundance	coll_species_richness	coll_nymphs_abundance	coll_shannon
0	BE_001	0.0000	0	0.0000	0.0000
1	BE_002	0.0000	0	0.0000	0.0000
2	BE_003	0.0000	0	0.0000	0.0000

8.- Datos de Secuenciación (Bacterias, Hongos, Eucariotas, Oomycetes, Cercozoa)

Las tablas ASV crudas son muy anchas (bacterias: 6798 ASVs, eucariotas: 4077 ASVs). Se calculan características de resumen por muestra:

- **Riqueza de ASVs:** número de ASVs con al menos 1 lectura.
- **Lecturas totales:** profundidad de secuenciación total.

Las tablas completas de ASVs se mantienen separadas por si se necesitan en análisis posteriores.

```
In [14]: def summarise_seq(filepath, sheet, sample_col='SAMPLE_ID', prefix=''):
    """
    Lee una tabla ASV y devuelve por sitio:
    - {prefix}asv_richness : número de ASVs con ≥1 lectura
    - {prefix}total_reads : profundidad de secuenciación total
    - {prefix}shannon : H' (ln) calculado desde la matriz de cuentas
    """
    df = pd.read_excel(filepath, sheet_name=sheet)
    asv_cols = [c for c in df.columns if c != sample_col]

    df[asv_cols] = df[asv_cols].apply(pd.to_numeric, errors='coerce').fillna(0)

    df[prefix + 'asv_richness'] = (df[asv_cols] > 0).sum(axis=1)
    df[prefix + 'total_reads'] = df[asv_cols].sum(axis=1)
    df[prefix + 'shannon'] = _shannon_matrix(df[asv_cols])

    df['SITE_ID'] = df[sample_col].str.extract(r'^([A-Z]{2}_\d{3})')

    return (
        df.groupby('SITE_ID')[[
            prefix + 'asv_richness',
            prefix + 'total_reads',
            prefix + 'shannon',
        ]]
        .mean(numeric_only=True)
        .reset_index()
    )

bac_sum = summarise_seq(DATA_DIR + 'DD2.2.12_BACTERIA_SEQ.xlsx',
                        'DD2.2.12_BACTERIA_SEQ', prefix='bac_')
fun_sum = summarise_seq(DATA_DIR + 'DD2.2.13_FUNGI_SEQ.xlsx',
                        'DD2.2.13_FUNGI_SEQ', prefix='fun_')
euk_sum = summarise_seq(DATA_DIR + 'DD2.2.14_EUKARYOTE_SEQ.xlsx',
                        'DD2.2.14_18S_SEQ', prefix='euk_')
oomy_sum = summarise_seq(DATA_DIR + 'DD2.2.15_OOMYCETES_CERCOZOA_SEQ.xlsx',
                        'OOMYCETE_SEQ', prefix='oomy_')
cerc_sum = summarise_seq(DATA_DIR + 'DD2.2.15_OOMYCETES_CERCOZOA_SEQ.xlsx',
                        'CERCOZOAN_SEQ', prefix='cerc_')

for name, df in [('Bacteria', bac_sum),
                 ('Fungi', fun_sum),
                 ('Eukaryotes', euk_sum),
                 ('Oomycetes', oomy_sum),
                 ('Cercozoa', cerc_sum)]:
    sh_col = [c for c in df.columns if c.endswith('_shannon')][0]
    print(f'{name:12s}: {df.shape} sites: {df["SITE_ID"].nunique()}'
          f' shannon max={df[sh_col].max():.3f}')
```

```
Bacteria : (391, 4) sites: 391 shannon max=6.090
Fungi : (390, 4) sites: 390 shannon max=1.088
Eukaryotes : (391, 4) sites: 391 shannon max=4.876
Oomycetes : (398, 4) sites: 398 shannon max=4.147
Cercozoa : (398, 4) sites: 398 shannon max=5.444
```

```
In [15]: # Aplicar normalización snake_case al dataset fusionado completo
_df_rename = {c: _to_snake(c) for c in df.columns if _to_snake(c) != c}
df = df.rename(columns=_df_rename)

print(f'Columnas renombradas en df tras la fusión: {len(_df_rename)}')
for old, new in sorted(_df_rename.items()):
    print(f' {old} → {new}')
```



```
print(f'\nForma de df tras normalización: {df.shape}')
print('Verificando que no quedan nombres con mayúsculas (excepto acrónimos controlados)...')
still_mixed = [c for c in df.columns if c != c.lower()]
if still_mixed:
    print(f' [WARN] {len(still_mixed)} columnas aún con mayúsculas: {still_mixed}')
else:
    print(' Todas las columnas en snake_case...')
```

Columnas renombradas en df tras la fusión: 1
"SITE_ID" → "site_id"

Forma de df tras normalización: (398, 4)
Verificando que no quedan nombres con mayúsculas (excepto acrónimos controlados)...
Todas las columnas en snake_case...

9.- Características Raster Europeas (Extracción de Valores por Punto)

Todos los rasters están en el estándar **ETRS89-LAEA**, el cual es equivalente a EPSG:3035. Para cada sitio SOB4ES (`lat`, `lon`) en WGS84, el punto se reprojecta al CRS nativo del raster y se extrae el valor del píxel correspondiente.

Tipo	Capas	Resolución	Nodata
Físicas	bulk_density, clay/sand/silt, textura, WHC	500 m	-3.4×10 ³⁸ (float32)
Químicas	CN, K, N, P, pH	500 m	-3.4×10 ³⁸ (float32)
Arsénico	LUCAS-median.tif	250 m	-3.4×10 ³⁸ (float32)
Carbono orgánico OCTOP	octop_insp.tif	1000 m	-3.4×10 ³⁸ (float32)
Metales pesados	Cu (500 m), Ni, Pb, Zn (1000 m)	mixto	-3.4×10 ³⁸ (float32)
Zonas ambientales	eu_env_zones_2018_esdac.tif	100 m	0 (uint8)
Uso del suelo	eu_land_cover_2018_corine.tif	100 m	-128 (int8)
Tipo de suelo WRB	eu_soil_type_wrb_2006_esdac.tif	1000 m	0 (uint8)

Nota sobre el dataset OCTOP

Los archivos `arc.dir`, `arc0000.dat`, `arc0000.nit`, `arc0001.dat`, `arc0001.nit`, `dblnd.adf`, `hdr.adf`, `sta.adf`, `w001001.adf`, `w001001x.adf` y `metadata.xml` son el **formato ESRI GRID original** del dataset OCTOP (Carbono Orgánico en el Horizonte Superior de Suelos en Europa, ESDAC).

El archivo `octop_insp.tif` es la conversión a GeoTIFF con el CRS correctamente embebido (EPSG:3035). Ambos dan valores idénticos, por lo tanto **se usa el TIF**.

Nota sobre el ESRI GRID sin CRS embebido

El `hdr.adf` abre correctamente con rasterio pero devuelve `CRS=None`. El `metadata.xml` confirma que la proyección es **ETRS_1989_LAEA** (= EPSG:3035). En caso de querer el GRID directamente, se puede obtener mediante la asignación directa del CRS de la siguiente forma:

```
with rasterio.open('hdr.adf') as src:
    crs = CRS.from_epsg(3035) # asignado manualmente desde metadata.xml
    xs, ys = rio_transform(WGS84, crs, [lon], [lat])
```

```
In [ ]: # Helpers de extracción raster
#
# Valores nodata en los archivos EU:
# float32 → -3.4028e+38 (o -3.4000e+38 en LUCAS-median)
# int32 → -2147483648 (soil_texture.tif)
# uint8 → 0 (env_zones, soil_type WRB - 0 no es clase válida)
# int8 → -128 (CORINE land cover)

def _apply_nodata_mask(vals: np.ndarray, nodata) → np.ndarray:
    """Convierte nodata y el centinela float32 a NaN en un array."""
    vals = vals.astype(float)
    if nodata is not None:
        vals[np.isclose(vals, float(nodata), rtol=1e-3)] = np.nan
        vals[vals < -1e35] = np.nan # centinela float32
    return vals

def batch_query_raster(tif_path: str,
                       lats: np.ndarray,
                       lons: np.ndarray) → np.ndarray:
    """
    Extrae los valores de un GeoTIFF para N puntos (lat, lon) en WGS84.
    Abre el archivo UNA SOLA VEZ y usa rasterio.sample() para leer
    solo los píxeles necesarios - mucho más eficiente que un bucle por fila.

    Devuelve un array float64 de longitud N; NaN en puntos nodata/fuera de bounds.
    """
    with rasterio.open(tif_path) as src:
        # Reprojectar todas las coords de golpe al CRS nativo del raster
        xs, ys = rio_transform(WGS84, src.crs, lons.tolist(), lats.tolist())

        # rasterio.sample() lee solo los tiles necesarios (eficiente en COG/GeoTIFF)
        # Devuelve un generador de arrays de 1 elemento por punto
        sampled = np.array(
            [v[0] for v in src.sample(zip(xs, ys), indexes=1, masked=False)],
            dtype=float
        )

        # Marcar puntos fuera de la extensión del raster como NaN
        bounds = src.bounds
        out_of_bounds = (
            (np.array(xs) < bounds.left) | (np.array(xs) > bounds.right) |
            (np.array(ys) < bounds.bottom) | (np.array(ys) > bounds.top)
        )
        sampled[out_of_bounds] = np.nan
```

```

        return _apply_nodata_mask(sampled, src.nodata)

# Diccionarios de lookup (código entero → etiqueta textual)

# 1. Clases de textura USDA (soil_texture.tif, códigos 1-12)
USDA_TEXTURE = {
    1: 'Clay',          2: 'Silty Clay',      3: 'Silty Clay Loam',  4: 'Sandy Clay',
    5: 'Sandy Clay Loam', 6: 'Clay Loam',      7: 'Silt',            8: 'Silt Loam',
    9: 'Sandy Loam',      10: 'Loamy Sand',    11: 'Sand',           12: 'Loam',
}
dbf_tex_codes = {int(r['Value']) for r in DBF(EU_DIR + 'eu_2015_esdac/soil_texture.vat.dbf')}
for code in dbf_tex_codes - set(USDA_TEXTURE):
    USDA_TEXTURE[code] = f'Class_{code}'
print(f'Textura USDA: {len(USDA_TEXTURE)} clases')

# 2. Zonas Ambientales EEA 2018 (códigos 1-15, nodata=0)
#EEA_ENV_ZONES (Nombre completo)
# 1: 'Alpine North',      2: 'Boreal',          3: 'Nemoral',
# 4: 'Atlantic North',    5: 'Alpine South',    6: 'Continental',
# 7: 'Atlantic Cental',   8: 'Pannonian',       9: 'Lusitanian',
# 10: 'Anatolian',        11: 'Mediterranean Mountains', 12: 'Mediterranean North',
# 13: 'Mediterranean South', 14: 'Macaronesia',    15: 'Arctic'
#
EEA_ENV_ZONES = {
    1: 'ALN', 2: 'BOR', 3: 'NEM',
    4: 'ALN', 5: 'ALS', 6: 'CON',
    7: 'ATC', 8: 'PAN', 9: 'LUS',
    10: 'ANA', 11: 'MDM', 12: 'MDN',
    13: 'MDS', 14: 'MAC', 15: 'ARC'
}
print(f'Zonas EEA: {len(EEA_ENV_ZONES)} zonas')

# 3. CORINE Land Cover 2018 (nodata=-128)
# El raster almacena códigos COMPACTOS 1-44 (int8).
# La leyenda CSV tiene columnas: code(CLC 3 dígitos), r, g, b, alpha, label.
# Se construye el lookup compacto→etiqueta en dos pasos:
# (a) leer CLC→label desde el CSV
# (b) mapear compacto→CLC→label

_corine_df = pd.read_csv(
    EU_DIR + 'eu_land_cover_2018_corine/eu_land_cover_2018_corine_legend.csv',
    header=None, names=['code', 'r', 'g', 'b', 'alpha', 'label'])
_clc_labels = dict(zip(_corine_df['code'].astype(int), _corine_df['label'].str.strip()))

_COMPACT_TO_CLC = {
    1:111, 2:112, 3:121, 4:122, 5:123, 6:124, 7:131, 8:132, 9:133,
    10:141, 11:142, 12:211, 13:212, 14:213, 15:221, 16:222, 17:223,
    18:231, 19:241, 20:242, 21:243, 22:244, 23:311, 24:312, 25:313,
    26:321, 27:322, 28:323, 29:324, 30:331, 31:332, 32:333, 33:334,
    34:335, 35:411, 36:412, 37:421, 38:422, 39:423,
    40:511, 41:512, 42:521, 43:522, 44:523, 48:999,
}
CORINE_LABELS = {
    compact: _clc_labels.get(clc, f'CLC_{clc}')
    for compact, clc in _COMPACT_TO_CLC.items()
}
print(f'CORINE: {len(CORINE_LABELS)} clases (códigos compactos 1-44 → etiquetas CLC)...')
print(f'Ejemplos: 18 → {CORINE_LABELS.get(18)}, 23 → {CORINE_LABELS.get(23)}, 12 → {CORINE_LABELS.get(12)}')

# 4. Tipo de Suelo WRB 2006 (códigos 1-30, nodata=0)
# Valores 1-6 = no-suelo (Urbano, Agua...); 7-30 = Grupos de Referencia WRB
_wrb_vat = pd.DataFrame(iter(DBF(EU_DIR + 'eu_soil_type_wrb_2006/eu_soil_type_wrb_2006_esdac.vat.dbf')))
_wrb_leg = pd.read_csv(EU_DIR + 'eu_soil_type_wrb_2006/eu_soil_type_wrb_2006_esdac_legend.csv')
_wrb_joined = _wrb_vat.merge(_wrb_leg, left_on='WRBLV1', right_on='code', how='left')
NON_SOIL_WRB = {1: 'Town', 2: 'Soil disturbed by man', 3: 'Water body',
                4: 'Marsh', 5: 'Glacier', 6: 'Rock outcrops', 30: 'No information'}
WRB_SOIL_LABELS = {}
for _, row in _wrb_joined.iterrows():
    val = int(row['VALUE'])
    WRB_SOIL_LABELS[val] = (
        NON_SOIL_WRB[val] if val in NON_SOIL_WRB
        else f'{row["WRBLV1"]} - {row["description"]}' if pd.notna(row.get('description'))
        else str(row['WRBLV1'])
    )
print(f'WRB suelo: {len(WRB_SOIL_LABELS)} clases...')

Textura USDA: 12 clases
Zonas EEA: 15 zonas
CORINE: 45 clases (códigos compactos 1-44 → etiquetas CLC)
Ejemplos: 18→Pastures, 23→Broad-leaved forest, 12→Non-irrigated arable land
WRB suelo: 30 clases

```

```

In [17]: # Registro de Capas Raster Europeas
# Formato: 'nombre_columna': (ruta_archivo, unidad, (límite_inf, límite_sup))
# Para añadir una nueva capa: agregar una línea y volver a ejecutar las celdas 28-31.
# Si el límite no tiene restricción, usar None.

```

```

EU_RASTERS = {
    # Propiedades físicas - 500 m (eu_2015_esdac)

    'eu_bulk_density'      : (EU_DIR + 'eu_2015_esdac/bulk_density.tif',      'g/cm³', (0.0, 3.0)),
    'eu_clay_content'      : (EU_DIR + 'eu_2015_esdac/clay_content.tif',      '%', (0.0, 100.0)),
    'eu_sand_content'      : (EU_DIR + 'eu_2015_esdac/sand_content.tif',      '%', (0.0, 100.0)),
    'eu_silt_content'      : (EU_DIR + 'eu_2015_esdac/silt_content.tif',      '%', (0.0, 100.0)),
    'eu_soil_texture_class' : (EU_DIR + 'eu_2015_esdac/soil_texture.tif',      'USDA', (1, 12)),
    'eu_water_holding_capacity' : (EU_DIR + 'eu_2015_esdac/water_holding_capacity.tif', 'vol fr', (0.0, None)),

```

```

# Propiedades químicas - 500 m (eu_2019_chemical_esdac)

'eu_CN_ratio'      : (EU_DIR + 'eu_2019_chemical_esdac/CN.tif',      'ratio', (0.0, None)),
'eu_K'             : (EU_DIR + 'eu_2019_chemical_esdac/K.tif',       'mg/kg', (0.0, None)),
'eu_N'             : (EU_DIR + 'eu_2019_chemical_esdac/N.tif',       'g/kg', (0.0, None)),
'eu_P'             : (EU_DIR + 'eu_2019_chemical_esdac/P.tif',       'mg/kg', (0.0, None)),
'eu_pH'            : (EU_DIR + 'eu_2019_chemical_esdac/pH.tif',      'pH', (0.0, 14.0)),

# Arsénico - 250 m (eu_arsenic)

'eu_As' : (EU_DIR + 'eu_arsenic/LUCAS-median.tif',      '%', (0.0, 100.0)),

# Carbono orgánico - 1000 m (octop_insp_directory)
# Archivo fuente original: ESRI GRID (hdr.adf + w001001.adf + resto de .adf)
# Se usa el GeoTIFF exportado (octop_insp.tif) porque tiene CRS embebido.
# Ambos producen valores idénticos (verificado en BE_001: 3.2933%).

'eu_organic_carbon_octop' : (EU_DIR + 'octop_insp_directory/octop_insp.tif',      '%', (0.0, 100.0)),

# Metales pesados - 500 m (eu_copper)

'eu_Cu' : (EU_DIR + 'eu_copper/copper_map_fill.tif',      'mg/kg', (0.0, None)),

# Metales pesados - 1000 m (eu_heavy_metals)

'eu_Ni' : (EU_DIR + 'eu_heavy_metals/Ni_EU27.tif',        'mg/kg', (0.0, None)),
'eu_Pb' : (EU_DIR + 'eu_heavy_metals/Pb_EU27.tif',        'mg/kg', (0.0, None)),

# Zinc - 1000 m (eu_zinc)

'eu_Zn' : (EU_DIR + 'eu_zinc/zinc.tif',                    'mg/kg', (0.0, None)),

# Categóricas: Zonas Ambientales - 100 m (eu_env_zones_2018_esdac)
# Códigos enteros 1-14; nodata=0 (uint8). Etiquetas en EEA_ENV_ZONES.

'eu_env_zone' : (EU_DIR + 'eu_env_zones_2018_esdac/eu_env_zones_2018_esdac.tif',      'código', (1, 14)),

# Categóricas: Uso del Suelo - 100 m (eu_land_cover_2018_corine)
# Códigos enteros 111-999; nodata=-128 (int8). Etiquetas en CORINE_LABELS.

'eu_land_cover' : (EU_DIR + 'eu_land_cover_2018_corine/eu_land_cover_2018_corine.tif',      'código', (1, 48)),

# Categóricas: Tipo de Suelo WRB - 1000 m (eu_soil_type_wrb_2006)
# Códigos enteros 1-30; nodata=0 (uint8). Etiquetas en WRB_SOIL_LABELS.
# Valores 1-6 = no-suelo (Urbano, Agua, etc.); 7-30 = Grupos de Referencia WRB.

'eu_soil_type_wrb' : (EU_DIR + 'eu_soil_type_wrb_2006/eu_soil_type_wrb_2006_esdac.tif',      'código', (1, 30)),

# Formato para añadir nuevas capas (si se obtienen nuevos datos)
# 'eu_XXX': (EU_DIR + 'XXX.tif', 'unidad', (lim_inf, lim_sup)),
# Al meter datos nuevos es recomendable ejecutar de nuevo desde la celda 28.
}

print(f'{len(EU_RASTERS)} capas raster europeas registradas')
for col, (path, unit, bounds) in EU_RASTERS.items():
    print(f' {col:35s} {unit:7s} límites={bounds}')

```

```

20 capas raster europeas registradas
eu_bulk_density      g/cm³      límites=(0.0, 3.0)
eu_clay_content      %          límites=(0.0, 100.0)
eu_sand_content      %          límites=(0.0, 100.0)
eu_silt_content      %          límites=(0.0, 100.0)
eu_soil_texture_class USDA      límites=(1, 12)
eu_water_holding_capacity vol fr límites=(0.0, None)
eu_CN_ratio          ratio      límites=(0.0, None)
eu_K                 mg/kg      límites=(0.0, None)
eu_N                 g/kg       límites=(0.0, None)
eu_P                 mg/kg      límites=(0.0, None)
eu_pH                pH         límites=(0.0, 14.0)
eu_As                %          límites=(0.0, 100.0)
eu_organic_carbon_octop %          límites=(0.0, 100.0)
eu_Cu                mg/kg      límites=(0.0, None)
eu_Ni                mg/kg      límites=(0.0, None)
eu_Pb                mg/kg      límites=(0.0, None)
eu_Zn                mg/kg      límites=(0.0, None)
eu_env_zone          código     límites=(1, 14)
eu_land_cover        código     límites=(1, 48)
eu_soil_type_wrb     código     límites=(1, 30)

```

In [18]: # Extracción vectorizada: abre cada raster UNA sola vez
Rendimiento: 20 aperturas de archivo (una por capa) en vez de abrir por cada lectura.
rasterio.sample() lee solo los tiles necesarios, algo que es eficiente en GeoTIFF/COG.

```

lats = sites['latitude'].to_numpy()
lons = sites['longitude'].to_numpy()

eu_records = {}
for col, (path, unit, bounds) in tqdm(EU_RASTERS.items(),
                                     desc='Extracción rasters EU',
                                     total=len(EU_RASTERS)):
    eu_records[col] = batch_query_raster(path, lats, lons)

df_eu = pd.DataFrame(eu_records, index=sites.index)

# Añadir site_id como columna para que el merge posterior funcione
df_eu.insert(0, 'site_id', sites['site_id'].values)

print(f'Características EU - forma: {df_eu.shape}')
df_eu.head(3)

```

```

Extracción rasters EU:  0%|          | 0/20 [00:00<?, ?it/s]
Características EU - forma: (428, 21)

```

Out[18]:	site_id	eu_bulk_density	eu_clay_content	eu_sand_content	eu_silt_content	eu_soil_texture_class	eu_water_holding_capacity	eu_CN_ratio	eu_K	eu_I
0	BE_001	1.0610	9.4843	61.2884	29.2273	12.0000	0.0877	12.3327	175.3472	3.644
1	BE_002	1.0610	9.4843	61.2884	29.2273	12.0000	0.0877	12.3327	175.3472	3.644
2	BE_003	1.0945	11.0963	59.4709	29.4328	12.0000	0.0815	12.4574	141.9444	2.822

In [19]: # Conversión de códigos numéricos categóricos a etiquetas de texto.

```
def _replace_codes_with_labels(df, code_col, lookup, new_col_name):
    """Sustituye la columna numérica por su etiqueta textual y la renombra."""
    if code_col not in df.columns:
        return
    df[new_col_name] = (
        df[code_col].dropna().astype(int).map(lookup)
        .reindex(df.index)
    )
    df.drop(columns=[code_col], inplace=True)

# eu_soil_texture_class (1-12) → eu_soil_texture (texto USDA)
_replace_codes_with_labels(df_eu, 'eu_soil_texture_class', USDA_TEXTURE, 'eu_soil_texture')

# eu_env_zone (1-15) → eu_env_zone (abreviatura EEA, sobreescibir)
_replace_codes_with_labels(df_eu, 'eu_env_zone', EEA_ENV_ZONES, 'eu_env_zone')

# eu_land_cover (compacto 1-44) → eu_land_use (texto CLC)
_replace_codes_with_labels(df_eu, 'eu_land_cover', CORINE_LABELS, 'eu_land_use')

# eu_soil_type_wrb (1-30) → eu_soil_type_wrb (texto WRB, sobreescibir)
_replace_codes_with_labels(df_eu, 'eu_soil_type_wrb', WRB_SOIL_LABELS, 'eu_soil_type_wrb')

# Resumen de conversión
for col in ['eu_soil_texture', 'eu_env_zone', 'eu_land_use', 'eu_soil_type_wrb']:
    if col in df_eu.columns:
        n_valid = df_eu[col].notna().sum()
        top3 = df_eu[col].value_counts().head(3).to_dict()
        print(f'{col}: {n_valid} válidos - top3: {top3}')
```

eu_soil_texture: 373 válidos - top3: {'Sandy Loam': 113, 'Silt Loam': 74, 'Loam': 63}

eu_land_use: 377 válidos - top3: {'Non-irrigated arable land': 94, 'Complex cultivation patterns': 45, 'Pastures': 45}

In [20]: # Informe de cobertura por capa

```
cobertura = (df_eu.notna().sum() / len(df_eu) * 100).round(1)
cobertura = cobertura.rename('cobertura_%').to_frame()
cobertura['n_validos'] = df_eu.notna().sum()
cobertura['n_nulos'] = df_eu.isna().sum()

print('Cobertura por capa raster EU:')
print(cobertura.sort_values('cobertura_%').to_string())

# Capas con cobertura insuficiente (<80%), revisar rutas o extensión geográfica
bajas = cobertura[cobertura['cobertura_%'] < 80]
if len(bajas):
    print(f'\n[WARN] {len(bajas)} capas con cobertura < 80%:')
    print(bajas.to_string())
```

Cobertura por capa raster EU:

	cobertura_%	n_validos	n_nulos
eu_CN_ratio	72.9000	312	116
eu_K	72.9000	312	116
eu_P	72.9000	312	116
eu_N	72.9000	312	116
eu_pH	72.9000	312	116
eu_As	75.9000	325	103
eu_Ni	76.2000	326	102
eu_Pb	76.2000	326	102
eu_Zn	76.2000	326	102
eu_Cu	76.4000	327	101
eu_soil_texture	87.1000	373	55
eu_bulk_density	87.4000	374	54
eu_water_holding_capacity	87.4000	374	54
eu_silt_content	87.9000	376	52
eu_sand_content	87.9000	376	52
eu_clay_content	87.9000	376	52
eu_land_use	88.1000	377	51
eu_organic_carbon_octop	88.1000	377	51
site_id	100.0000	428	0

[WARN] 10 capas con cobertura < 80%:

	cobertura_%	n_validos	n_nulos
eu_CN_ratio	72.9000	312	116
eu_K	72.9000	312	116
eu_N	72.9000	312	116
eu_P	72.9000	312	116
eu_pH	72.9000	312	116
eu_As	75.9000	325	103
eu_Cu	76.4000	327	101
eu_Ni	76.2000	326	102
eu_Pb	76.2000	326	102
eu_Zn	76.2000	326	102

In [21]: # Verificación de límites de dominio y recorte

Aplica los límites físicos definidos en EU_RASTERS para detectar y corregir
valores fuera de rango (e.g. pH > 14, arcilla > 100%).

```
flag_eu = []
for col, (path, unit, (lo, hi)) in EU_RASTERS.items():
    if col not in df_eu.columns:
        continue
    mask = pd.Series(False, index=df_eu.index)
    if lo is not None:
        mask |= df_eu[col] < lo
```

```

if hi is not None:
    mask |= df_eu[col] > hi
n = mask.sum()
if n:
    flag_eu.append({'columna': col, 'unidad': unit, 'n_fuera_rango': n,
                    'rango_valido': f'[{lo}, {hi}]'})
    df_eu[col] = df_eu[col].clip(
        lower=lo if lo is not None else -np.inf,
        upper=hi if hi is not None else np.inf
    )

if flag_eu:
    print('Valores fuera de rango detectados y recortados:')
    print(pd.DataFrame(flag_eu).to_string(index=False))
else:
    print('Todos los límites de dominio superados...')

```

Todos los límites de dominio superados...

10.- Fusión de Todas las Fuentes

Todos los datasets se unen a la tabla base `sites` mediante `SITE_ID` (left join conserva los 428 sitios).

```

In [22]: df = sites.copy()

plan_fusion = [
    ('fisicos_sitio',      phys_site),
    ('quimicos_sitio',    chem_site),
    ('quimicos_parcela',  chem_plot_agg),
    ('diversidad_alfa',    alpha_div), # solo nematode_shannon
    ('macrofauna',        macro_agg),
    ('lombrices_resumen', ew_summary_final),
    ('oribatida',         orib_sum),
    ('mesostigmata',      meso_sum),
    ('collembola',        coll_sum),
    ('bacterias',         bac_sum),
    ('hongos',            fun_sum),
    ('eucariotas',        euk_sum),
    ('oomycetes',         oomy_sum),
    ('cercosozoa',        cerc_sum),
    ('eu_rasters',        df_eu),
]

print(f'Tabla base (sites): {df.shape}')

for etiqueta, right in plan_fusion:
    if right is None or (hasattr(right, 'empty') and right.empty):
        print(f' [SKIP] {etiqueta}: vacio - omitido')
        continue

    # Detectar dinámicamente la columna de sitio
    col_sitio = [c for c in right.columns if c.lower() == 'site_id']
    if not col_sitio:
        print(f' [WARN] {etiqueta}: sin columna de identificación de sitio - omitido')
        continue

    col_sitio = col_sitio[0]
    right = right.copy()
    if col_sitio != 'site_id':
        right = right.rename(columns={col_sitio: 'site_id'})

    # Normalizar nombres del right a snake_case (excepto site_id que ya está OK)
    right.columns = ['site_id' if c == 'site_id' else _to_snake(c) for c in right.columns]

    df['site_id'] = df['site_id'].astype(str).str.replace('.', '', regex=False)
    right['site_id'] = right['site_id'].astype(str).str.replace('.', '', regex=False)

    antes = df.shape[1]
    df = df.merge(right, on='site_id', how='left', suffixes=('', f'_{etiqueta}'))
    print(f' + {etiqueta:25s} → +{df.shape[1]-antes:4d} cols (total: {df.shape[1]})')

print(f'\nForma total fusionada: {df.shape}')
assert len(df) == len(sites), f'Pérdida de filas errónea: {len(sites)} → {len(df)}'
print('Integridad de filas verificada...\n')

# Normalizar TODO df a snake_case de una vez
df.columns = [_to_snake(c) for c in df.columns]

# Purga de columnas innecesarias
# 1. Columnas redundantes
cols_redundantes = ['earthworm_density_m2']

# 2. Artefactos de identificación / pipeline (ya en snake_case)
cols_artefactos = [
    'ew_species_richness_calc', 'nuid_nan', 'sample_id',
    'eu_textura_suelo_nombre', 'eu_zona_ambiental_nombre', 'eu_tipo_suelo_wrb_nombre',
]

# 3. Capas EU con cobertura <80%
cols_eu_baja_corr = ['eu_bulk_density', 'eu_cu', 'eu_k', 'eu_n', 'eu_ni', 'eu_pb']

# 4. Macrofauna individual taxa - keep only the 3 summary columns
macro_taxa_individual = [
    c for c in df.columns
    if c.startswith('macro_')
    and c not in ('macro_total_abundance', 'macro_order_richness', 'macro_shannon')
]

todas_a_eliminar = (cols_redundantes + cols_artefactos
                    + cols_eu_baja_corr + macro_taxa_individual)
columnas_finales_a_tirar = [c for c in todas_a_eliminar if c in df.columns]
df = df.drop(columns=columnas_finales_a_tirar, errors='ignore')

```

```
print(f'Eliminadas {len(columnas_finales_a_tirar)} columnas innecesarias...')
print(f'    Macrofauna taxa individuales eliminadas: {len(macro_taxa_individual)}')

# 5. Artefactos Excel (Unnamed)
cols_unnamed = [c for c in df.columns if 'unnamed' in c]
df = df.drop(columns=cols_unnamed, errors='ignore')
print(f'Eliminadas {len(cols_unnamed)} columnas de artefacto estructural de Excel ('Unnamed')...')

print(f'\nDatos fusionados correctamente...')
```

```
Tabla base (sites): (428, 13)
+ fisicos_sitio          → + 6 cols (total: 19)
+ quimicos_sitio        → + 9 cols (total: 28)
+ quimicos_parcela      → + 3 cols (total: 31)
+ diversidad_alfa       → + 1 cols (total: 32)
+ macrofauna           → + 25 cols (total: 57)
+ lombrices_resumen     → + 3 cols (total: 60)
+ oribatida            → + 3 cols (total: 63)
+ mesostigmata          → + 3 cols (total: 66)
+ collembola           → + 4 cols (total: 70)
+ bacterias            → + 3 cols (total: 73)
+ hongos               → + 3 cols (total: 76)
+ eucariotas           → + 3 cols (total: 79)
+ oomycetes            → + 3 cols (total: 82)
+ cercozoa             → + 3 cols (total: 85)
+ eu_rasters           → + 18 cols (total: 103)
```

```
Forma total fusionada: (428, 103)
Integridad de filas verificada...
```

```
Eliminadas 29 columnas innecesarias...
    Macrofauna taxa individuales eliminadas: 22
Eliminadas 0 columnas de artefacto estructural de Excel ('Unnamed')...
```

```
Datos fusionados correctamente...
```

11.- Limpieza de Datos

11.1.- Auditoría de Valores Perdidos

```
In [23]: missing = (
            df.isnull().sum()
            .rename('n_missing')
            .to_frame()
        )
missing['pct_missing'] = (missing['n_missing'] / len(df) * 100).round(1)
missing = missing[missing['n_missing'] > 0].sort_values('pct_missing', ascending=False)

print(f'Columnas totales      : {df.shape[1]}')
print(f'Columnas con NaNs : {len(missing)}')
print(f'Columnas completamente vacías : {(missing["pct_missing"] == 100).sum()}\n')

# Mostrar distribución de datos perdidos
bins = [0, 5, 25, 50, 75, 100]
labels = ['<5%', '5-25%', '25-50%', '50-75%', '75-100%']
missing['bucket'] = pd.cut(missing['pct_missing'], bins=bins, labels=labels, right=True)
print(f'distribución de valores perdidos: {missing["bucket"]}')

```

Columnas totales : 74
Columnas con NaNs : 59
Columnas completamente vacías : 0

distribución de valores perdidos: orib_total_abundance 25-50%

orib_shannon	25-50%
orib_species_richness	25-50%
eu_cn_ratio	25-50%
eu_p	25-50%
eu_ph	25-50%
aggregate_stability	5-25%
eu_as	5-25%
eu_zn	5-25%
earthworm_shannon	5-25%
earthworm_abundance	5-25%
earthworm_richness	5-25%
macro_shannon	5-25%
macro_total_abundance	5-25%
macro_order_richness	5-25%
coll_species_richness	5-25%
coll_nymphs_abundance	5-25%
coll_total_abundance	5-25%
coll_shannon	5-25%
eu_soil_texture	5-25%
eu_water_holding_capacity	5-25%
eu_silt_content	5-25%
eu_clay_content	5-25%
eu_sand_content	5-25%
eu_land_use	5-25%
eu_organic_carbon_octop	5-25%
fun_shannon	5-25%
fun_total_reads	5-25%
fun_asv_richness	5-25%
euk_shannon	5-25%
bac_total_reads	5-25%
euk_asv_richness	5-25%
bac_asv_richness	5-25%
bac_shannon	5-25%
euk_total_reads	5-25%
cerc_total_reads	5-25%
oomy_shannon	5-25%
oomy_asv_richness	5-25%
oomy_total_reads	5-25%
cerc_shannon	5-25%
cerc_asv_richness	5-25%
sand_content	5-25%
silt_content	5-25%
clay_content	5-25%
soil_moisture	<5%
bulk_density	<5%
soil_ph	<5%
cu	<5%
k	<5%
as	<5%
ni	<5%
zn	<5%
pb	<5%
mo	<5%
p	<5%
meso_species_richness	<5%
meso_total_abundance	<5%
nematode_shannon	<5%
meso_shannon	<5%

Name: bucket, dtype: category
Categories (5, object): ['<5%' < '5-25%' < '25-50%' < '50-75%' < '75-100%']

```
In [24]: # Eliminar columnas por encima del umbral de valores perdidos
MISS_THRESHOLD = 70.0 # %

drop_cols = missing[missing['pct_missing'] >= MISS_THRESHOLD].index.tolist()
print(f'Dropping {len(drop_cols)} columns (>={MISS_THRESHOLD}% missing)')

df_clean = df.drop(columns=drop_cols)
print(f'Shape after drop: {df_clean.shape}')
```

Dropping 0 columns (>70.0% missing)
Shape after drop: (428, 74)

```
In [25]: # Limpieza de datos

# 1. Filas sin coordenadas (eliminar)
# 2. Conteos ecológicos (ew_*, macro_*, orib_*, meso_*, coll_*, nuid_*, uvigo_*)
# En caso de ausencia de datos, rellenar con 0s.
# 3. Variables continuas con < 5% NaN: Imputar con mediana (sin indicador)
# 4. Variables continuas con 5-50% NaN: Añadir columna _was_missing (0/1)
# + imputar con mediana. El modelo puede aprender que la ausencia de dato
# también es informativa.
# 5. Variables continuas con > 50% NaN: Ya eliminadas en la celda anterior
# 6. Categóricas → imputar con moda

# Eliminar filas sin coordenadas
n_before = len(df_clean)
df_clean = df_clean.dropna(subset=['latitude', 'longitude'])
print(f'Filas sin coordenadas eliminadas: {n_before - len(df_clean)}...')

# Prefijos ecológicos: NaN = ausencia de especie se convierte a 0
ECO_PREFIXES = ['ew_', 'macro_', 'orib_', 'meso_', 'coll_', 'nuid_', 'uvigo_']
eco_cols = [c for c in df_clean.select_dtypes(include=np.number).columns
             if c.startswith(ECO_PREFIXES)]
df_clean[eco_cols] = df_clean[eco_cols].fillna(0)
print(f'Conteos ecológicos → 0: {len(eco_cols)} columnas...')

# Variables continuas restantes
num_cols = [c for c in df_clean.select_dtypes(include=np.number).columns
```

```

        if c not in eco_cols]

miss_pct = df_clean[num_cols].isnull().mean() * 100
cols_indicator = miss_pct[(miss_pct ≥ 5) & (miss_pct ≤ 50)].index.tolist()
cols_median_only = miss_pct[(miss_pct > 0) & (miss_pct < 5)].index.tolist()

# Añadir indicadores de missingness para columnas 5-50%
for col in cols_indicator:
    df_clean[col + '_was_missing'] = df_clean[col].isna().astype(int)
print(f'Indicadores _was_missing añadidos: {len(cols_indicator)} columnas...')

# Imputar todas las numéricas restantes con mediana
all_num = df_clean.select_dtypes(include=np.number).columns
medians = df_clean[all_num].median()
df_clean[all_num] = df_clean[all_num].fillna(medians)

# Imputar categóricas con moda
cat_cols = df_clean.select_dtypes(include='object').columns.tolist()
for col in cat_cols:
    mode_val = df_clean[col].mode()
    if len(mode_val):
        df_clean[col] = df_clean[col].fillna(mode_val[0])

remaining_na = df_clean.isnull().sum().sum()
print(f'NaN restantes tras imputación: {remaining_na}...')
print(f'Shape tras limpieza: {df_clean.shape}...')

```

Filas sin coordenadas eliminadas: 0...
 Conteos ecológicos → 0: 13 columnas...
 Indicadores _was_missing añadidos: 32 columnas...
 NaN restantes tras imputación: 0...
 Shape tras limpieza: (428, 106)...

11.2.- Comprobaciones de Sentido Físico

```

In [26]: # Límites físicos para datos de suelo y medioambiente
DOMAIN_BOUNDS = {
    'latitude'      : (-90, 90),
    'longitude'     : (-180, 180),
    'clay_content'  : (0, 100),
    'silt_content'  : (0, 100),
    'sand_content'  : (0, 100),
    'Bulk density'  : (0, 3),    # g/cm³
    'Soil moisture' : (0, 1),    # fraction
    'aggregate_stability': (0, 1),
    'soil_pH'       : (0, 14),
    'plot_Total_organic_C': (0, 100), # %
    'plot_Total_N'  : (0, 100), # %
    'Total_Plant_cover' : (0, 100), # %
    # Alpha diversity indices must be non-negative
    **{c: (0, None) for c in df_clean.columns if 'Shannon' in c or 'richness' in c.lower()},
}

flag_report = []
for col, (lo, hi) in DOMAIN_BOUNDS.items():
    if col not in df_clean.columns:
        continue
    mask = pd.Series([False] * len(df_clean), index=df_clean.index)
    if lo is not None:
        mask |= df_clean[col] < lo
    if hi is not None:
        mask |= df_clean[col] > hi
    n_out = mask.sum()
    if n_out:
        flag_report.append({'column': col, 'n_invalid': n_out, 'valid_range': f'[{lo}, {hi}]'})
        df_clean[col] = df_clean[col].clip(
            lower=lo if lo is not None else -np.inf,
            upper=hi if hi is not None else np.inf
        )

if flag_report:
    print('Valores fuera de rango truncados:')
    print(pd.DataFrame(flag_report).to_string(index=False))
else:
    print('Todas las comprobaciones de dominio realizadas correctamente...')

```

Valores fuera de rango truncados:
 column n_invalid valid_range
 aggregate_stability 36 [0, 1]

```

In [27]: # Verificar suma de texturas: arcilla + limo + arena ≈ 100%
tex = ['clay_content', 'silt_content', 'sand_content']
if all(c in df_clean.columns for c in tex):
    texture_sum = df_clean[tex].sum(axis=1)
    bad = ((texture_sum < 95) | (texture_sum > 105)).sum()
    print(f'Suma de texturas fuera del [95-105]?: {bad} filas...')
    if bad:
        df_clean[tex] = df_clean[tex].div(texture_sum, axis=0).mul(100)
        print(' Re-normalizado al 100%...')

```

Suma de texturas fuera del [95-105]?: 40 filas...
 Re-normalizado al 100%...

11.3.- Detección de Outliers

```

In [28]: def iqr_outlier_mask(series: pd.Series, factor: float = 3.0) → pd.Series:
    Q1, Q3 = series.quantile(0.25), series.quantile(0.75)
    IQR = Q3 - Q1
    return (series < Q1 - factor * IQR) | (series > Q3 + factor * IQR)

outlier_report = []
for col in df_clean.select_dtypes(include=np.number).columns:

```



```

n = iqr_outlier_mask(df_clean[col]).sum()
if n:
    outlier_report.append({'column': col, 'n_outliers': n,
                          'pct': round(n / len(df_clean) * 100, 1)})

df_outlier_rpt = (pd.DataFrame(outlier_report)
                  .sort_values('n_outliers', ascending=False))

print(f'Columns with IQR outliers (factor=3): {len(df_outlier_rpt)}')
display(df_outlier_rpt.head(20).to_string(index=False))

# Marcar filas que son outliers en cualquier columna numérica
df_clean['outlier_flag'] = False
for col in df_clean.select_dtypes(include=np.number).columns:
    df_clean['outlier_flag'] |= iqr_outlier_mask(df_clean[col])

print(f'\nFilas marcadas como outliers: {df_clean["outlier_flag"].sum()} '
      f'({df_clean["outlier_flag"].mean()*100:.1f}%)')

```

```

Columns with IQR outliers (factor=3): 60
'
column n_outliers pct\n
eu_zn_was_missing 102 23.8000\n earthworm_richness_was_missing 101 23.6000\n eu_as_was_missing 103 24.1000\n
101 23.6000\n earthworm_shannon_was_missing 101 23.6000\n coll_nymphs_abundance 62 14.5000\n eu_water
_holding_capacity_was_missing 54 12.6000\n eu_clay_content_was_missing 52 12.1000\n eu_silt_content_was
_missing 52 12.1000\n eu_sand_content_was_missing 52 12.1000\n eu_organic_carbon_octop_was_missing 51
11.9000\n total_plant_cover 43 10.0000\n fun_total_reads_was_missing 38 8.9000\n fun
_asv_richness_was_missing 38 8.9000\n fun_shannon_was_missing 38 8.9000\n bac_shannon_was_mis
sing 37 8.6000\n bac_total_reads_was_missing 37 8.6000\n bac_asv_richness_was_missing 37 8.6
000\n euk_shannon_was_missing 37 8.6000\n euk_total_reads_was_missing 37 8.6000'
Filas marcadas como outliers: 341 (79.7%)

```

11.4.- Filas Duplicadas

```

In [29]: n_exact = df_clean.duplicated().sum()
n_coord = df_clean.duplicated(subset=['latitude', 'longitude']).sum()

print(f'Filas duplicadas exactas      : {n_exact}')
print(f'Coordenadas duplicadas       : {n_coord}')

if n_exact:
    df_clean = df_clean.drop_duplicates()
    print(f'Eliminar duplicados exactos. Formato: {df_clean.shape}')

```

```

Filas duplicadas exactas      : 0
Coordenadas duplicadas       : 37

```

12.- Estandarización y Codificación

12.1.- Codificación de Variables Categóricas

```

In [30]: from sklearn.preprocessing import LabelEncoder
import joblib

# Codificar variables categóricas para label_encoders.pkl.
# Los _enc NO se incluyen en ningún CSV de salida (ni clean ni model_ready).
CATEGORICAL_COLS = [
    'country',
    'pedoclimatic_region',
    'soil_type',
    'land_use_type',
    'land_use_intensity',
    'dominant_vegetation',
    'eu_land_use',      # texto CLC (reemplaza eu_land_cover numérico)
    'eu_soil_texture',  # texto USDA (reemplaza eu_soil_texture_class numérico)
    'eu_env_zone',      # abreviatura EEA (reemplaza código numérico)
    'eu_soil_type_wrb', # texto WRB (reemplaza código numérico)
]

label_encoders = {}
for col in CATEGORICAL_COLS:
    if col not in df_clean.columns:
        print(f' [SKIP] {col} no encontrado en df_clean')
        continue
    le = LabelEncoder()
    le.fit(df_clean[col].astype(str))
    label_encoders[col] = le
    print(f' {col}: {len(le.classes_)} clases')

joblib.dump(label_encoders, OUT_DIR + 'label_encoders.pkl')
print(f'\nEncoders guardados → {OUT_DIR}label_encoders.pkl')

```

```

country: 12 clases
pedoclimatic_region: 9 clases
soil_type: 22 clases
land_use_type: 7 clases
land_use_intensity: 3 clases
dominant_vegetation: 36 clases
eu_land_use: 21 clases
eu_soil_texture: 8 clases
[SKIP] eu_env_zone no encontrado en df_clean
[SKIP] eu_soil_type_wrb no encontrado en df_clean

```

Encoders guardados → output/label_encoders.pkl

12.2.- Estandarización Numérica (StandardScaler)

```

In [31]: from sklearn.preprocessing import StandardScaler

# Columnas excluidas del escalado:
# - Coordenadas e IDs (no son features)
# - Variables categóricas nominales (ya codificadas con _enc)

```

```
# - Indicadores _was_missing (son binarios 0/1, no escalar)
# - Columnas _enc y _was_missing
EXCLUDE_FROM_SCALING = [
    'latitude', 'longitude', 'site_id', 'sampling_date', 'outlier_flag',
    # Nominales: se usan como enteros directamente (no escalar)
]

scale_cols = [
    c for c in df_clean.select_dtypes(include=np.number).columns
    if c not in EXCLUDE_FROM_SCALING
    and not c.endswith('_enc')
    and not c.endswith('_was_missing')
]

scaler = StandardScaler()
scaled_arr = scaler.fit_transform(df_clean[scale_cols])
df_scaled = pd.DataFrame(
    scaled_arr,
    columns=[c + '_z' for c in scale_cols],
    index=df_clean.index
)

joblib.dump(scaler, OUT_DIR + 'scaler.pkl')
print(f'Escaladas {len(scale_cols)} columnas numéricas')
print(f'Scaler guardado: {OUT_DIR}scaler.pkl')
```

Escaladas 61 columnas numéricas
Scaler guardado: output/scaler.pkl

```
In [32]: # Ensamblar el dataset final
df_final = pd.concat([df_clean.reset_index(drop=True),
                      df_scaled.reset_index(drop=True)], axis=1)

print(f'Final dataset: {df_final.shape[0]} sites × {df_final.shape[1]} columns...')
```

Final dataset: 428 sites × 168 columns...

12.3.- Catálogo de Columnas

```
In [33]: def infer_source(col):
    if col.startswith('eu_'):          return 'EU Raster'
    if col.startswith('macro_'):       return 'Macrofauna'
    if col.startswith('ew_'):          return 'Earthworms (combined)'
    if col.startswith('nuid_'):        return 'Earthworms (NUID_UCD raw)'
    if col.startswith('uvigo_'):       return 'Earthworms (UVI60 raw)'
    if col.startswith('orib_'):        return 'Oribatida'
    if col.startswith('meso_'):        return 'Mesostigmata'
    if col.startswith('coll_'):        return 'Collembola'
    if col.startswith('bac_'):         return 'Bacteria (16S)'
    if col.startswith('fun_'):         return 'Fungi (ITS)'
    if col.startswith('euk_'):         return 'Eukaryotes (18S)'
    if col.startswith('oomy_'):        return 'Oomycetes'
    if col.startswith('cerc_'):        return 'Cercozoa'
    if col.startswith('plot_'):        return 'Abiotic (plot-level)'
    if col in ['clay_content', 'silt_content', 'sand_content',
               'aggregate_stability', 'Bulk density', 'Soil moisture']: return 'Abiotic (physical)'
    if col in ['As', 'Cu', 'K', 'Mo', 'Ni', 'P', 'Pb', 'Zn', 'soil_pH']: return 'Abiotic (chemical)'
    if 'Shannon' in col or 'SHANNON' in col: return 'Alpha diversity'
    if col.endswith('_z'):              return 'Scaled (z-score)'
    if col.endswith('_enc'):            return 'Encoded (label)'
    return 'Site metadata'

catalogue = pd.DataFrame({
    'dtype' : df_final.dtypes,
    'n_missing' : df_final.isnull().sum(),
    'n_unique' : df_final.nunique(),
    'source' : [infer_source(c) for c in df_final.columns],
})

print('\nColumnas por fuente:')
print(catalogue.groupby('source').size().sort_values(ascending=False).to_string())
catalogue
```

Columnas por fuente:
source

Site metadata	35
EU Raster	32
Scaled (z-score)	20
Eukaryotes (18S)	9
Cercozoa	9
Bacteria (16S)	9
Oomycetes	9
Fungi (ITS)	9
Collembola	8
Abiotic (plot-level)	6
Mesostigmata	6
Macrofauna	6
Oribatida	6
Abiotic (physical)	4

Out[33]:

	dtype	n_missing	n_unique	source
site_id	object	0	428	Site metadata
country	object	0	12	Site metadata
pedoclimatic_region	object	0	9	Site metadata
site_locality	object	0	181	Site metadata
latitude	float64	0	342	Site metadata
...	...	...	...	...
eu_p_z	float64	0	227	EU Raster
eu_ph_z	float64	0	228	EU Raster
eu_as_z	float64	0	259	EU Raster
eu_organic_carbon_octop_z	float64	0	122	EU Raster
eu_zn_z	float64	0	242	EU Raster

168 rows × 4 columns

13.- Exportación

Varios archivos de salida:

- sob4es_clean_vx.csv : Dataset limpio completo (escala original, legible).
- sob4es_model_ready_vx.csv : Solo columnas _z (escaladas) y _enc (codificadas), listo para ML.
- sob4es_imputation_flags_vx.csv : Dataset que contiene solamente las columnas _was_missing.
- scaler.pkl / label_encoders.pkl : Transformadores para aplicar a nuevos datos.

In [34]:

```
import pandas as pd

VERSION = 'v15'
ID_COLS = ['site_id', 'latitude', 'longitude']

# 1.- Ensamblar matrices
df_final = pd.concat([df_clean.reset_index(drop=True),
                      df_scaled.reset_index(drop=True)], axis=1)

# Seguridad: purgar cualquier _enc_z residual
enc_z_cols = [c for c in df_final.columns if c.endswith('_enc_z') or c.endswith('_enc')]
df_final = df_final.drop(columns=enc_z_cols, errors='ignore')

# 2.- Dataset de indicadores de imputación
missing_cols_detected = [c for c in df_clean.columns if c.endswith('_was_missing')]
if missing_cols_detected:
    df_imputation_flags = df_clean[['site_id'] + missing_cols_detected].copy()
    flags_path = OUT_DIR + f'sob4es_imputation_flags_{VERSION}.csv'
    df_imputation_flags.to_csv(flags_path, index=False)
    print(f'Flags: {flags_path}')
    print(f' ({df_imputation_flags.shape[0]} filas × {df_imputation_flags.shape[1]} cols)')

# 3.- Dataset limpio (escala original, sin _enc, sin _was_missing)
cols_sin_wm = [c for c in df_clean.columns if not c.endswith('_was_missing')]
df_clean_export = df_clean[cols_sin_wm]
clean_path = OUT_DIR + f'sob4es_clean_{VERSION}.csv'
df_clean_export.to_csv(clean_path, index=False)
print(f'Dataset limpio: {clean_path}')
print(f' ({df_clean_export.shape[0]} filas × {df_clean_export.shape[1]} cols)')

# 4.- Dataset para el modelo: solo _z + outlier_flag (sin _enc)
z_cols = [c for c in df_final.columns if c.endswith('_z')]
flag_col = ['outlier_flag'] if 'outlier_flag' in df_final.columns else []

model_df = df_final[ID_COLS + flag_col + z_cols].copy()
model_path = OUT_DIR + f'sob4es_model_ready_{VERSION}.csv'
model_df.to_csv(model_path, index=False)
print(f'Dataset modelo: {model_path}')
print(f' ({model_df.shape[0]} filas × {model_df.shape[1]} cols)')
print(f' {len(z_cols)} columnas _z')

print(f'\nExportación {VERSION} completada.')
print(f'Sitios: {df_clean_export.shape[0]} | ')
print(f'Features limpias: {df_clean_export.shape[1]} | Features modelo: {model_df.shape[1]}')

Flags: output/sob4es_imputation_flags_v15.csv
(428 filas × 33 cols)
Dataset limpio: output/sob4es_clean_v15.csv
(428 filas × 75 cols)
Dataset modelo: output/sob4es_model_ready_v15.csv
(428 filas × 65 cols)
61 columnas _z

Exportación v15 completada.
Sitios: 428 |
Features limpias: 75 | Features modelo: 65
```